

① Design Pattern :- These are reusable solutions to common problems that occur during software development. Design Pattern help developer create software that is more maintainable, scalable and easier to understand.

○ There are 3 main type of design pattern :-

1. Creational Pattern

2. Structural Pattern

3. Behavioral Pattern

① Creational Pattern :- It focuses on creation of objects. It helps in making object creation more flexible and efficient.

② Structural Pattern :- It focuses on how classes and objects are composed to form larger structures. They help to add new functionality without

having to modify the entire System.
They ensure that if one part of a system changes, whole System does not change with it.

⑦ Behavioral Pattern :- It focuses on how objects communicate and interact with each other, assign responsibilities to them and make sure they all have Synchronized information.

① Different types of Creational Pattern:-

1. Factory Pattern :- It is a pattern that provides a template that can be used to create objects. It is used in situations where type of objects required varies and needs to be specified in each case.

It helps with object creation logic abstraction and easier extraction.

eg:- You want to create different type of payments without exposing instantiation logic.

interface Payment

{ void pay()

}

class CreditCard implements Payment

{ p.v pay()

{ s.o.p ("Paid with Credit Card");

}

}

class PayPal implements Payment

{ p.v pay()

{ s.o.p ("Paid with PayPal");

}

class Payment Factory

{ static Payment getPayment (String
type)

{
return type.equals("paypal") ?

new PayPal() : new CreditCard();

}

}

// usage

Payment p = Payment.getPayment
("paypal");

p.pay();

2. Constructor Pattern :- It is a class based pattern that uses constructors present in the class to create specific type of objects.

eg:- If you need to create a simple objects with known value.

```
class Item
{
    String name;
    double price;
    Item (String name, double price)
    {
        this.name = name;
        this.price = price;
    }
}

Item i = new Item ("Phone", 800.00);
```

3. Singleton Pattern :- It helps

with having a global access to a single shared instance.

eg:- Parking Lot Configuration

(Only one instance of configuration should exist globally)


```
class ParkingLot
```

```
{    p.s ParkingLot instance = new ParkingLot();
```

```
    private ParkingLot()
```

```
    public static ParkingLot getInstance()
```

```
    {    return instance ;
```

```
    }
```

```
    public void Show ()
```

```
    {    s.o.p (" Single Parking Lot Instance ");
```

```
    }
```

// Usage

```
ParkingLot lot = ParkingLot.getInstance();  
lot.show();
```

4. Builder Pattern :- It helps in building complex objects using simpler objects.

eg! - Shopping Cart order with optional Fields

(If you want to create an object with optional / custom fields fluently)

class Order

{

String item ; int qty ;

boolean gift ;

static class Builder

{

String item ;

int qty ;

boolean gift ;

}

Builder qtx (int q)

{
 qtx = q ;

 return this ;

}

Builder gift (boolean g)

{
 gift = g ;

 return this ;

}

Order build()

{

Order o = new Order();

o.item = item;

o.qty = qty;

o.gift = gift;

return o;

}

```
public void Show ()
```

```
{ S.O.P ( item + "n" + qty +  
    " Gift:" + gift );
```

```
}
```

```
// Usage
```

```
Order O = new Order.Builder  
    ("Book").qty(2).gift(true).  
    build();  
O.Show();
```

5. Abstract Pattern:- It helps with creating related objects without knowing exact classes..

eg:- Vehicle Factory

(You want to create families of related objects (like Bike / Car) without binding code to specific classes.


```
interface Vehicle
```

```
{    void park();
```

```
}
```

```
class Bike implements Vehicle
```

```
{    p. v park()
```

```
{    S.O.P(" Bike Parked"),
```

```
}
```

```
}
```

class Car implements Vehicle

{ p.v park()

{ s.o.p("Car parked");

}

interface Vehicle Factory

{ vehicle create();

}

```
class CarFactory implements VehicleFactory
{
    public Vehicle create ()
    {
        return new Car ();
    }
}
```

```
class BikeFactory implements VehicleFactory
{
    public Vehicle create ()
    {
        return new Bike ();
    }
}
```

// usage

VehicleFactory factory = new BikeFactory()
factory.createPart();

① Different Types of Structural Pattern :-

1. Decorator Pattern (Add features dynamically)

② Wraps an object to add new behavior without modifying the original class.

① When to use :- Add responsibilities to objects at runtime (eg :- adding gift wrap or discount to orders).

eg! Shopping Cart Item With Gift Wrap

```
interface Item
```

```
{ String getDescription();  
}
```

```
class Book implements Item
```

```
{ public String getDescription()  
  { return "Book";  
  }  
}
```

class GiftWrapDecorator implements Item

{ private Item item;

GiftWrapDecorator (Item item)

{ this.item = item;

} public String getDescription ()

{ return item.getDescription +
" + Gift Wrap ";

}

// usage

```
Item book = new GiftWrapDecorator  
            (new Book());
```

```
S.o.p (book.getDescription());
```

// output :- Book +

Gift wrap.

2. Facade Pattern :- (Simply Complex Systems)

- It provides a unified interface to a set of subsystems.
- When to use :- To hide a complex logic behind a simple Interface.

eg:- Shopping Cart Check out

```
class Inventory
```

```
{ void check ()
```

```
{ S.O.P. ("Inventory checked");
```

```
}
```

```
}
```

```
class Payment Gateway
```

```
{ void process ()
```

```
{ S.O.P. ("Payment processed");
```

```
}
```

```
class Notification
```

```
{ void send()
```

```
{ S.O.P ("Notification Sent");
```

```
}
```

```
}
```

```
class CheckoutFacade {
```

```
Inventory inventory = new Inventory();
```

```
PaymentGateway payment = new PaymentGateway();
```

```
Notification notify = new Notification();
```

```
void checkout()
```

```
{  
    inventory.check(),  
    payment.process(),  
    notify.send(),  
}
```

```
}
```

// Usage

```
new CheckoutFacade - checkout();
```

- o It helps with making subsystem easier to use.